Part 5 — Jenkins Credentials & Security

Complete DevOps Interview Notes | Beginner to Practical Project Level

These notes cover Jenkins Credentials IDs, AWS authentication, IAM roles, secret handling, permissions, and the principle of least privilege. Every topic includes five headings, with practical examples and scenario-based interview questions and answers.

1. Jenkins Credentials IDs

1. Definition — What is it?

Jenkins Credentials are a secure way to store sensitive information required by a pipeline, such as:

- Usernames and passwords
- GitHub Personal Access Tokens (PATs)
- AWS access keys and secret keys
- SSH private keys
- Secret text, API tokens, and certificates

A **Credentials ID** is a unique identifier Jenkins uses to retrieve a stored credential. The pipeline references the ID instead of directly writing the secret in the Jenkinsfile. For example, **aws-jenkins-credentials** could be the ID of a stored AWS credential.

2. Purpose — Why do we use it?

- To avoid hardcoding passwords and access keys in source code.
- To centralize credential management.
- To allow pipelines to access secrets without exposing them in the Jenkinsfile.
- To simplify credential rotation and updates.
- To control which jobs and users can access credentials.

3. Working — How does it work?

1. Add a credential: Manage Jenkins > Credentials > Add Credentials
2. Assign a Credentials ID. Example: github-pat or aws-jenkins-credentials
3. Reference it in the pipeline using credentials() or withCredentials()
4. Use the secret securely to authenticate to AWS, GitHub, or another service

Jenkins makes the credential available to the relevant pipeline step or scope, depending on the binding used. Masking may help hide secrets in console logs, but it is not a guarantee against every form of leakage.

4. Practical Example — How do we use it in a project?

Suppose your Jenkins pipeline runs Terraform commands to create AWS infrastructure. You have stored an AWS credential in Jenkins with the ID **aws-jenkins-credentials**. For a pipeline using the AWS Credentials plugin:

```
withCredentials([
    [$class: 'AmazonWebServicesCredentialsBinding',
     credentialsId: 'aws-jenkins-credentials']
]) {
    bat 'aws sts get-caller-identity'
    bat 'terraform plan'
}
```

This binding exposes AWS credentials to the commands inside the block. It assumes the AWS Credentials plugin is installed and the stored credential is configured correctly.

Important: For AWS workloads running on EC2, prefer an IAM role and temporary credentials where possible rather than long-lived IAM access keys.

5. Scenario-Based Interview Questions and Answers

Q1. You need to connect Jenkins to AWS. Would you hardcode the access key in the Jenkinsfile?

Answer: No. I would store the credential in Jenkins Credentials and reference its Credentials ID. For an EC2-based Jenkins agent, I would prefer an attached IAM role with temporary credentials when practical.

Q2. Your Jenkins pipeline says Could not find credentials with ID 'aws-jenkins-credentials'. How would you troubleshoot it?

Answer:

1. Open Manage Jenkins > Credentials.
2. Confirm that the credential exists.
3. Verify the ID spelling and capitalization.
4. Check whether the credential is stored in the correct scope or folder.
5. Confirm that the required plugin is installed and the credential type is supported.

Q3. A credential is available in Jenkins, but a pipeline cannot access it. What could be the reason?

Answer: The credential may be in a different folder or scope, the job may lack access, the credential type may be incompatible with the binding, or a required plugin may be missing. I would check these before recreating the credential.

2. AWS Authentication in Jenkins

1. Definition — What is it?

AWS authentication is the process by which Jenkins proves its identity to AWS before calling AWS services. For example, a Jenkins pipeline running Terraform may need to authenticate to AWS to create an EC2 instance, configure an EKS cluster, or manage S3 state.

Authentication answers **“Who are you?”** Authorization answers **“What are you allowed to do?”**

2. Purpose — Why do we use it?

- To allow Jenkins to communicate securely with AWS.
- To run AWS CLI and Terraform commands.
- To control access to services such as EC2, S3, EKS, and IAM.
- To identify which AWS identity performed an operation.
- To avoid unauthorized access to cloud resources.

3. Working — How does it work?

Jenkins can obtain AWS credentials in several ways:

Method	How it works	Typical recommendation
IAM role attached to EC2	AWS supplies temporary credentials to the instance	Preferred for EC2-hosted Jenkins
IAM access key and secret key	Stored in Jenkins Credentials and injected into a build	Use only when necessary; rotate and protect them

Method	How it works	Typical recommendation
Federated authentication	Uses an identity provider and temporary role credentials	Excellent for supported CI/CD setups
Environment/profile credentials	Obtained from a configured credential provider or profile	Must be configured securely on the agent

A typical pipeline flow is:

1. Jenkins starts a build.
2. The build obtains credentials through the configured authentication method.
3. The AWS CLI or Terraform provider uses those credentials.
4. AWS validates the identity.
5. IAM policies determine which operations are permitted.

4. Practical Example — How do we use it in a project?

In a Windows Jenkins and Terraform environment, a basic AWS identity check can be run before Terraform commands:

```
stage('Verify AWS Authentication') {
    steps {
        bat 'aws sts get-caller-identity'
    }
}
```

If authentication succeeds, AWS returns information such as the account ID, ARN, and user or role identifier.

If you use a Jenkins-stored access key, bind it securely first. If Jenkins runs on an EC2 instance with a suitable IAM role, the AWS CLI can normally use the role's temporary credentials automatically, provided no conflicting configuration overrides them.

Never print access keys or secret keys to the console. **aws sts get-caller-identity** is useful because it verifies the identity without requiring you to display the secret.

5. Scenario-Based Interview Questions and Answers

Q1. Your Terraform pipeline works locally but fails in Jenkins with Unable to locate credentials. What would you check?

Answer: I would check how the local machine authenticates, then verify that the Jenkins agent has an equivalent supported authentication method. I would inspect the agent's IAM role, Jenkins credential binding, AWS profile configuration, and region settings. I would then run aws sts get-caller-identity from the same pipeline context.

Q2. Jenkins successfully authenticates to AWS, but Terraform cannot create an EC2 instance. Why?

Answer: Authentication may be successful while authorization is insufficient. I would inspect the IAM policies attached to the user or role, check the denied AWS action and resource, and review any applicable permission boundary, service control policy, or explicit deny.

Q3. How would you authenticate Jenkins running on an AWS EC2 instance?

Answer: I would attach a dedicated IAM role to the EC2 instance and grant it only the required permissions. The AWS CLI and supported SDKs can obtain temporary credentials through the instance's role, avoiding static access keys stored on disk.

3. IAM Roles for Jenkins

1. Definition — What is it?

An IAM role is an AWS identity that has a set of permissions and can be assumed by an authorized entity. Unlike a typical IAM user's long-term access keys, a role provides temporary security credentials when assumed.

For Jenkins, a common approach is to attach an IAM role to the EC2 instance hosting Jenkins or to the specific build agent that needs AWS access. Two important concepts:

- Trust policy: Defines who or what is allowed to assume the role.
- Permissions policy: Defines which AWS actions the role is allowed to perform.

2. Purpose — Why do we use it?

- To avoid storing long-lived AWS access keys on Jenkins.
- To provide temporary credentials.
- To restrict Jenkins to the AWS actions it needs.
- To improve auditability through AWS CloudTrail.
- To separate permissions for different pipelines or environments.

3. Working — How does it work?

For Jenkins running on EC2:

1. Create an IAM role for the Jenkins EC2 instance.
2. Configure its trust relationship to allow EC2 to assume the role.
3. Attach a least-privilege permissions policy.
4. Attach the role to the EC2 instance through an instance profile.
5. The AWS SDK or CLI obtains temporary credentials from the EC2 instance metadata service.
6. Jenkins runs AWS CLI or Terraform commands using those credentials.

For Jenkins running outside AWS, a federated approach such as OIDC-based role assumption may be more appropriate if the platform and plugins support it.

4. Practical Example — How do we use it in a project?

Imagine Jenkins runs Terraform to provision an EKS cluster. The pipeline needs permissions for specific AWS resources, but it should not have unrestricted administrator access. An illustrative permissions policy might allow read-only access to describe EC2 resources:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances",
        "ec2:DescribeVpcs",
        "ec2:DescribeSubnets"
      ],
      "Resource": "*"
    }
  ]
}
```

This is an illustrative read-only policy, not a complete policy for creating an EKS cluster. Provisioning EKS requires additional permissions for the specific services and resources involved.

In a real project, I would start from the actions Terraform actually requires, scope resources where AWS supports it, test the pipeline, and refine the policy based on access-denied errors and audit logs.

5. Scenario-Based Interview Questions and Answers

Q1. Your Jenkins server is running on EC2. How would you avoid storing AWS access keys in Jenkins?

Answer: I would attach a dedicated IAM role to the EC2 instance. The AWS CLI and Terraform provider can use temporary credentials from the instance role, provided no conflicting credential configuration is used. This reduces the need for static secrets.

Q2. Your Jenkins role can read EC2 instances but cannot create them. What would you do?

Answer: I would inspect the denied action in the error message and CloudTrail, then review the role's permissions and any explicit denies or permission boundaries. I would add only the necessary creation and supporting permissions, scoped to the relevant resources wherever possible.

Q3. What is the difference between an IAM role's trust policy and its permissions policy?

Answer: The trust policy determines who can assume the role. The permissions policy determines what an entity can do after assuming the role.

4. Secret Handling in Jenkins

1. Definition — What is it?

Secret handling means storing, accessing, transmitting, and using sensitive information securely during pipeline execution. Secrets include passwords, private keys, API tokens, database credentials, and cloud access keys.

Secret handling is more than hiding text in the console. A secure pipeline minimizes secret exposure throughout its entire lifecycle.

2. Purpose — Why do we use it?

- To prevent unauthorized access to cloud resources and applications.
- To protect source code repositories from leaked credentials.
- To reduce the risk of secrets appearing in build logs or artifacts.
- To support credential rotation and incident response.
- To meet organizational security requirements.

3. Working — How does it work?

A secure secret-management workflow follows these practices:

1. Store secrets in Jenkins Credentials or an approved secret manager such as AWS Secrets Manager.
2. Give access only to the required jobs, folders, or agents.
3. Inject secrets into the narrowest possible pipeline scope.
4. Avoid printing secrets or enabling verbose debugging around them.
5. Do not write secrets to source files, Terraform plans, artifacts, or workspace files unnecessarily.
6. Rotate exposed or expired secrets and review access logs.

Jenkins can mask certain known secret values in console output, but masking is not a substitute for secure pipeline design. A malicious or poorly written build can still expose a secret through encoding, file output, network calls, or other means.

4. Practical Example — How do we use it in a project?

Suppose a Jenkins pipeline must use a GitHub token to access a private repository or API. Store the token as a Secret text credential with the ID **github-pat**.

```
withCredentials([
    string(
        credentialsId: 'github-pat',
```

```
        variable: 'GITHUB_TOKEN'
    )
]) {
    bat 'some-command-that-uses-the-token'
}
```

Replace the placeholder command with a tool configured to read the token from the GITHUB_TOKEN environment variable. Do not put the token directly in the command string or print it.

For a shell agent, use the equivalent sh step and quote shell variables carefully. Be aware that environment variables can sometimes be inspected by processes running under the same operating-system account.

For Terraform: Remember that Terraform state files and saved plan files can contain sensitive values. Protect the backend, restrict access, encrypt storage, and avoid publishing these files as general build artifacts.

5. Scenario-Based Interview Questions and Answers

Q1. A developer accidentally commits an AWS secret key to GitHub. What steps would you take?

Answer:

1. Revoke or deactivate the exposed key immediately.
2. Investigate CloudTrail and other logs for possible misuse.
3. Replace the key if it is still needed, using a secure authentication method.
4. Remove the secret from the repository history where appropriate, while understanding that removal does not invalidate the leaked key.
5. Review how the secret was introduced and prevent recurrence with secret scanning and secure credential management.

Q2. Jenkins masks a secret in the console, but the secret appears in a generated file. Is this secure?

Answer: No. Console masking does not protect files. I would remove the secret from generated artifacts, restrict workspace access, clean up exposed copies, and rotate the secret if unauthorized access is possible.

Q3. Can I store AWS keys in variables.tf or terraform.tfvars?

Answer: No, I would not store credentials in Terraform configuration or commit them to Git. I would use an IAM role or another supported temporary credential method. If a secret variable is unavoidable, I would inject it securely and protect Terraform state and plan files because sensitive values can still be recorded there.

5. Jenkins Permissions and Access Control

1. Definition — What is it?

Jenkins permissions control what users, groups, and automation accounts can do within Jenkins. Examples include:

- View jobs and build history.
- Create or configure jobs.
- Start or cancel builds.
- Access credentials.
- Manage plugins, security settings, and Jenkins configuration.

Jenkins access control generally involves **authentication** (who the user is) and **authorization** (what that user can do).

2. Purpose — Why do we use it?

- To prevent unauthorized users from changing pipelines.
- To restrict access to sensitive credentials and configuration.

- To separate developer, tester, and administrator responsibilities.
- To prevent accidental production deployments.
- To support accountability and auditing.

3. Working — How does it work?

A Jenkins administrator configures security and permissions based on the organization's needs. Typical roles might be:

Role	Typical access
Jenkins administrator	Manage Jenkins configuration and security
Developer	View jobs and run permitted builds
Tester	Run test pipelines and review results
Deployment operator	Run approved deployment jobs
Read-only user	View permitted jobs and build logs

Jenkins supports authorization strategies through its security configuration and plugins. Exact permissions depend on the chosen strategy and installed plugins.

Important: Do not give every user administrator access simply to make a pipeline work.

4. Practical Example — How do we use it in a project?

Suppose your team uses Jenkins for an AWS Terraform project.

- Developers can view pipeline results and run validation jobs.
- Only approved users can run production deployment jobs.
- A small group of administrators can manage Jenkins configuration.
- AWS credentials are accessible only to the relevant pipeline or folder, as far as the selected credential scope and authorization configuration permit.

A production deployment should also have an approval process when required by your team's change-management policy.

5. Scenario-Based Interview Questions and Answers

Q1. A developer can log in to Jenkins but cannot configure a job. What would you check?

Answer: I would check the authorization strategy, the developer's assigned permissions or group membership, and any folder-level restrictions. I would grant the required permission only if it matches the developer's responsibilities.

Q2. Every developer currently has Jenkins administrator access. How would you improve security?

Answer: I would identify the tasks each role needs to perform, configure role-based or project-based permissions, remove unnecessary administrative access, and test the changes with representative accounts.

Q3. How would you prevent unauthorized production deployments?

Answer: I would restrict access to deployment jobs, use an approval gate where appropriate, protect credentials, validate the target environment, and maintain an audit trail of build and deployment activity.

6. Principle of Least Privilege

1. Definition — What is it?

The principle of least privilege means giving a user, service, process, or pipeline only the permissions needed to perform its assigned task—and no more. For example, a pipeline that only runs **terraform**

validate should not need permission to delete AWS resources. Least privilege applies to both Jenkins and AWS.

2. Purpose — Why do we use it?

- To reduce the impact of compromised credentials.
- To prevent accidental deletion or modification of resources.
- To reduce the risk of privilege escalation.
- To limit access to production environments.
- To make security reviews and audits easier.

3. Working — How does it work?

Use this approach when assigning permissions:

1. Identify the exact task the user or pipeline needs to perform.
2. Identify the required Jenkins permissions and AWS API actions.
3. Grant only those permissions, scoped to the relevant jobs, resources, or environments wherever possible.
4. Test the pipeline and investigate access-denied errors.
5. Review permissions periodically and remove unnecessary access.

For Terraform, the permissions needed for plan and apply can differ depending on the providers, data sources, and resources in the configuration. A plan may require read access, while an apply may require create, update, delete, and supporting IAM actions.

4. Practical Example — How do we use it in a project?

Consider your EKS Terraform pipeline. A safer separation of responsibilities could be:

- Validation pipeline: Checks formatting and configuration without AWS modification permissions when possible.
- Plan pipeline: Uses an identity with the read and supporting permissions required to inspect infrastructure and produce a plan.
- Apply pipeline: Uses a more tightly controlled identity with the permissions required for approved changes.
- Production deployment: Requires restricted job access and an approval step where appropriate.

Do not assume that a single read-only policy will work for every Terraform plan. Terraform may need additional permissions to refresh state, inspect resources, or read data sources.

5. Scenario-Based Interview Questions and Answers

Q1. Your Terraform pipeline uses AdministratorAccess because it previously failed with permission errors. Is that a good solution?

Answer: No. Administrator access is overly broad for normal pipeline operations. I would identify the exact denied AWS actions, determine the resources involved, and create a more restrictive policy. I would test the pipeline and verify that it still performs the required tasks.

Q2. How would you reduce the risk of a Jenkins credential being compromised?

Answer: I would prefer temporary role-based credentials, restrict which jobs can access the credential, limit AWS permissions, avoid exposing secrets in logs or artifacts, rotate credentials when needed, and monitor activity through AWS CloudTrail.

Q3. A pipeline needs to create EKS clusters and IAM roles. Can you simply grant it eks:* and iam:*?

Answer: I would avoid broad wildcard permissions wherever practical. I would identify the required EKS and IAM actions and scope them to the appropriate resources. IAM role creation and passing roles to AWS

services need particular care because they can create privilege-escalation risks.

7. Complete Practical Example — Secure Jenkins and Terraform Pipeline

The following is a simplified example for a Windows Jenkins agent running Terraform. It demonstrates validation, an AWS identity check, a plan, and a controlled apply. This example assumes:

- Terraform and the AWS CLI are installed on the agent.
- AWS authentication is configured securely.
- The Jenkins agent label exists.
- The Terraform working directory and backend are correctly configured.
- The deployment identity has only the required permissions.
- The saved plan is protected as sensitive data.

```
pipeline {
    agent { label 'windows-terraform' }

    options {
        timestamps()
        disableConcurrentBuilds()
    }

    parameters {
        choice(
            name: 'ACTION',
            choices: ['plan', 'apply'],
            description: 'Choose the Terraform action'
        )
    }

    environment {
        AWS_DEFAULT_REGION = 'us-east-1'
        TF_IN_AUTOMATION = 'true'
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Terraform Init') {
            steps {
                bat 'terraform init'
            }
        }

        stage('Terraform Validate') {
            steps {
                bat 'terraform fmt -check -recursive'
                bat 'terraform validate'
            }
        }

        stage('Verify AWS Identity') {
            steps {
                bat 'aws sts get-caller-identity'
            }
        }

        stage('Terraform Plan') {
            steps {
                bat 'terraform plan -out=tfplan'
            }
        }
```

```
        stage('Approval') {
            when {
                expression {
                    params.ACTION == 'apply'
                }
            }
            steps {
                input message: 'Approve the Terraform changes?',
                      ok: 'Approve'
            }
        }

        stage('Terraform Apply') {
            when {
                expression {
                    params.ACTION == 'apply'
                }
            }
            steps {
                bat 'terraform apply -input=false tfplan'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully.'
        }

        failure {
            echo 'Pipeline failed. Check the build logs.'
        }

        always {
            echo 'Pipeline execution finished.'
        }
    }
}
```

Security improvements to apply before using it in production

1. Use an EC2 IAM role or supported federated authentication instead of static AWS keys when possible.
2. Protect the saved tfplan file and Terraform state. Do not publish them to unrestricted artifacts.
3. Restrict who can approve and run the apply stage. For stronger separation of duties, use a protected deployment job and appropriate authorization controls.
4. Use a remote backend with encryption, restricted access, and state locking.
5. Keep production credentials and permissions separate from development where appropriate.
6. Review the Terraform configuration and plan before approving destructive or high-impact changes.

The example does not itself configure authentication or enforce who is allowed to approve. Those controls must be configured in Jenkins and AWS.

8. Quick Interview Revision

Topic	Key point to remember
Credentials ID	Reference to a secret stored in Jenkins
AWS authentication	Establishes the AWS identity used by a pipeline
IAM role	Provides temporary credentials to an authorized entity
Trust policy	Controls who can assume an IAM role

Topic	Key point to remember
Permissions policy	Defines the actions allowed by the role
Secret handling	Protect secrets throughout their lifecycle
Jenkins permissions	Control who can access and modify Jenkins resources
Least privilege	Grant only the access needed for a specific task
AWS CloudTrail	Helps investigate AWS API activity
Terraform state	Can contain sensitive information and needs protection

9. Practice Checklist

- [ ] I can explain Jenkins Credentials IDs and credential bindings.
- [ ] I can explain IAM users versus IAM roles.
- [ ] I can verify AWS authentication using sts get-caller-identity.
- [ ] I know why secrets must not be hardcoded in Jenkinsfiles.
- [ ] I can explain Jenkins authorization and job permissions.
- [ ] I can apply least privilege to a Terraform pipeline.
- [ ] I know how to respond when an AWS secret is leaked.
- [ ] I can explain why Terraform state and plan files need protection.

Interview tip: When answering security scenario questions, explain four things: how you identify the problem, how you investigate it, how you fix it securely, and how you prevent it from happening again. This shows practical DevOps and production-support thinking.